VEXORA

Esports Tournament Platform

Complete Product, Technical, Security & Operations Blueprint

A real-time web + Android/iOS-ready ecosystem for players, teams, verified organisers and administrators — now with an MCP-assisted, phase-by-phase build plan, extended features and a hardened security model.

PLAYER APP	ORGANISER APP	ADMIN WEB
Discover games, join tournaments, track wallet, stats and rankings	**Create/manage matches, submit results and communicate**	**Approve organisers, manage users, wallet, codes, disputes and analytics**

Important launch principle: design Vexora as a skill-based esports/tournament service, not a betting or wagering service. Entry fees, cash prizes, wallet conversion and promotional mechanics can be regulated differently by jurisdiction. Obtain legal advice for India and each target state/country before enabling real-money features.

Contents

1. Executive Summary . . . 3
2. Product Roles & Permissions . . . 4
3. User Journey . . . 5
4. Tournament Module . . . 5
5. Game & Rules Engine . . . 6
6. Team System . . . 7
7. Player Performance & Leaderboards . . . 7
8. Internal Currency & Redeem-Code System . . . 8
9. Payments, Prize Pools & Financial Controls . . . 9
10. Organiser Verification & Trust . . . 9
11. Anti-Cheat, Anti-Fraud & Dispute System . . . 9
12. Admin Desktop Dashboard . . . 10
13. Mobile Application . . . 11
14. Real-Time Architecture . . . 11
15. Recommended Technical Architecture . . . 12
16. Database Design . . . 12
17. API Structure . . . 13
18. Security Requirements . . . 14
19. Anti-Abuse Controls . . . 14
20. Notifications & Communication . . . 15
21. Ratings & Reputation . . . 15
22. Analytics . . . 15
23. Support System . . . 16
24. Legal, Compliance & Policy Layer . . . 16
25. Trust Features That Make Users Comfortable . . . 17
26. Important Edge Cases . . . 17
27. Development Roadmap . . . 18
28. MVP Scope vs Later Features . . . 18
29. Testing Plan . . . 19
30. Suggested Folder Structure . . . 19
31. Core Business Rules to Lock Before Coding . . . 20
32. Example End-to-End Tournament . . . 20
33. What Makes This More Than a Tournament App . . . 20
34. Final Recommended Product Architecture . . . 21
35. Launch Checklist . . . 21
36. Conclusion . . . 22
37. MCP Toolchain for Building Vexora . . . 23

38. MCP Configuration & Safe-Use Rules . . . 24
39. Phase-by-Phase Build Plan . . . 25
40. MCP-to-Phase Matrix . . . 35
41. Extended Feature Set (Vexora Additions) . . . 36
42. Gamification & Retention . . . 37
43. Organiser Productivity Tools . . . 37
44. Vexora Brand & Design System (Stitch-driven) . . . 37
45. DevOps, CI/CD & Environments . . . 38
46. Observability, SLOs & Incident Response . . . 38
47. Extended Security Architecture . . . 39
48. Privacy & Data Protection . . . 40
49. Performance & Scalability . . . 41
50. Risk Register . . . 41
51. Sprint Cadence & Working Agreement . . . 42
52. Final Definition of Done for Vexora v1 . . . 42

How to read this document. Sections 1-36 are the original blueprint, fully preserved and branded for Vexora, with small security and feature enhancements marked "Vexora addition". Sections 37-52 are new: the MCP toolchain, the phase-by-phase build plan (with the exact MCP servers to use in each phase), extended features, a deeper security model and operational playbooks.

1. Executive Summary

Vexora is a multi-sided esports tournament ecosystem. Players primarily use the mobile application, organisers can use the mobile application and web interface, and administrators operate a secure desktop-first web dashboard. The system supports tournament discovery, registration, team management, match operations, result submission, leaderboards, performance analytics, internal credits, redeem codes, payout requests, organiser verification, disputes, notifications and real-time updates.

The central trust problem is important: a player needs confidence that an organiser will actually conduct the tournament and distribute the announced prize, while the organiser needs confidence that players will follow the rules and that fraudulent activity can be investigated. Therefore, trust, evidence, audit logs, identity verification, controlled money flows and dispute handling should be core features—not features added later.

Recommended product structure

Area	Primary user	Main interface
Administration	Admin / moderator / finance staff	Desktop web
Tournament operations	Verified organiser	Mobile app + web
Participation	Player / team	Mobile app + web
Trust & safety	Admin + moderator	Desktop web + selected mobile actions
Analytics	Player / organiser / admin	Mobile + web
Real-time events	All relevant users	WebSockets / push notifications

Core design goals

- Transparent tournament information: rules, entry conditions, schedule, prize structure, organiser identity and status.
- Strong organiser verification before the organiser can host public tournaments.
- Team-based and individual performance tracking with game-specific statistics.
- A controlled internal credit/ledger system instead of editing balances directly.
- Every important financial, moderation and tournament action creates an audit record.
- Real-time tournament status, match results, notifications and leaderboard updates.
- Fraud/dispute workflows based on evidence rather than location-based punishment or assumptions.
- Privacy-by-design: collect only information necessary for identity, safety, payments and compliance.
- Vexora addition: an MCP-assisted, phase-by-phase build so every phase is shippable, tested and reviewable before the next begins.
- Vexora addition: security as a default (secure-by-design, zero-trust admin, signed evidence, tamper-evident audit chain).

2. Product Roles & Permissions

Role	Can do	Cannot do
Player	Profile, join tournaments, team, wallet, stats, submit evidence, disputes	Approve own payout, approve organisers, edit official results
Team Captain	Manage roster, register team, receive team notices, confirm line-up	Change official match result without authorised process
Organiser	Create tournaments, schedule matches, verify players, submit results, upload evidence	Change wallet ledger, approve self, bypass dispute/admin control
Moderator	Review reports, evidence, chat, match disputes; temporary actions	Create financial credits outside authorised workflow
Finance/Admin	Codes, wallet reconciliation, payouts, refunds, financial reports	Change competitive results without audit trail
Super Admin	All configuration, roles, bans, game rules, system controls	Should not silently alter historical financial/audit records
Support Agent (Vexora addition)	View tickets, limited user context, reply, escalate	See KYC documents, change balances, change results
Auditor (Vexora addition)	Read-only access to audit logs, ledger and reports	Any write action

Permission model

Use Role-Based Access Control (RBAC), plus resource-level permissions. For example, an organiser may edit only tournaments owned by that organiser. A moderator can view evidence but cannot generate redeem codes. A finance operator can process a payout but cannot approve the same payout request they created. High-risk actions should require step-up authentication and, where appropriate, a second administrator.

> **Vexora addition.** Enforce permissions twice: in the API layer (guards) and in the database layer using PostgreSQL Row-Level Security (RLS) for multi-tenant data such as organiser-owned tournaments and user-owned wallets. Add time-boxed "break-glass" admin access that is logged, alerted and automatically expires.

3. User Journey

Player journey

- Install app -> create account -> verify phone/email -> choose favourite games -> create or join team.
- Open Discover -> choose game -> filter by entry requirement, date, format, prize pool and organiser status.
- Open tournament -> read complete rules, schedule, prize distribution, eligibility, entry fee/credit requirement and organiser profile.
- Join -> select team/slot -> confirm registration -> system locks required credits according to the tournament rules.
- Receive lobby/match instructions -> play -> submit required result/evidence if the tournament workflow requires it.
- Result verified -> statistics and leaderboard update -> prize/reward settlement is processed according to published rules.
- View match history, rank, performance dashboard, team history and notifications.

Organiser journey

- Apply as organiser -> submit required identity/contact/business/payment details -> admin review.
- After approval, create tournament draft -> select game, format, rules, schedule, capacity and prize structure.
- Submit for publication -> system validates mandatory fields and financial/prize constraints -> admin/moderation approval if configured.
- Open registrations -> manage participants and brackets -> publish lobby/match information at the correct time.
- Conduct match -> record results and upload evidence -> players receive result notification.
- Handle normal result confirmation or dispute -> finalise -> platform calculates leaderboard and settlement.
- Review organiser analytics: tournaments hosted, completion rate, disputes, participant ratings and settlement history.

Admin journey

- Dashboard -> monitor users, organisers, tournaments, wallet liability, pending payouts, disputes, suspicious activity and system health.
- Review organiser applications and verification evidence.
- Create/manage games, rule templates, tournament policies, prize configurations and system settings.
- Generate redeem codes in controlled batches with value, expiry, usage limits and status.
- Review payout requests and evidence -> approve/reject/hold -> maintain audit trail.
- Investigate disputes/fraud -> preserve evidence -> issue proportionate account actions according to published policy.

4. Tournament Module

Tournament creation fields

Category	Fields
Identity	Tournament name, game, organiser, logo/banner, tournament ID
Format	Solo/Duo/Squad/Team, knockout, league, round-robin, Swiss, custom
Eligibility	Age/region restrictions where legally appropriate, rank requirement, team size, account requirements
Schedule	Registration open/close, check-in time, match start, rounds, timezone
Capacity	Minimum/maximum participants or teams, waitlist
Entry	Free or approved paid-entry model; required internal credits if applicable
Prize	Total prize pool, distribution, conditions, settlement status
Rules	Game-specific rules, prohibited behaviour, evidence rules, disconnect policy, tie-breaker
Operations	Lobby instructions, room code handling, result reporting, support contact
Trust	Organiser verification status, organiser history, published dispute/ratings information

Tournament states

```
DRAFT -> PENDING_REVIEW -> SCHEDULED -> REGISTRATION_OPEN -> CHECK_IN -> LIVE ->
RESULT_PENDING -> DISPUTED (optional) -> COMPLETED -> SETTLED -> ARCHIVED
```

Do not allow arbitrary status changes. Every transition should be validated by the backend and recorded in an immutable audit log.

Bracket and match engine

- Generate bracket slots from approved registrations.
- Support byes and seed rules.
- Track match state: scheduled, lobby-ready, live, result-submitted, confirmed, disputed, final.
- Prevent duplicate match submissions with idempotency keys.
- For team games, lock the roster at the defined cutoff.
- Keep both raw submitted results and the final adjudicated result.

5. Game & Rules Engine

Do not hard-code every game into the application. Create a configurable Game Definition system. Each game can define supported modes, team size, scoring fields, evidence requirements and ranking formula.

Game configuration	Example
Game	Free Fire / COD Mobile / BGMI / other supported title
Mode	Battle Royale / Clash / 1v1 / custom
Team size	Solo / Duo / Squad / 4-5 player team
Scoring	Placement points + kills + bonuses, or custom
Evidence	Screenshot, result screen, replay/video where supported
Tie-breakers	Placement, kills, time, head-to-head, predefined rule
Leaderboard	Seasonal, all-time, game-specific, role-specific

For each game, publish the exact scoring formula. Never change a live tournament's scoring formula silently. If an emergency correction is required, record the old and new version and notify affected users.

> **Vexora addition.** Store scoring formulas as versioned JSON schemas validated by a shared TypeScript/Dart package, and run the same scoring engine on server and (read-only preview) on client so players can verify how points were computed. Add a "scoring simulator" in the organiser app to test a formula on sample results before publishing.

6. Team System

- Team profile: team name, tag, logo, captain, members, substitute members, game and season.
- Team roles: owner/captain, manager, player, substitute.
- Invite/accept/leave workflow with roster limits.
- Roster lock before tournament start.
- Team history: tournaments, wins/placements, points, earnings/rewards where legally applicable, participation.
- Team rating should be based on transparent metrics; do not hide the calculation.
- Prevent duplicate competitive identities where the game/provider rules prohibit them.
- Vexora addition: in-game ID verification (player links their in-game UID; organiser can request a profile screenshot match).
- Vexora addition: transfer window rules (limit roster changes during a season) and a published team transfer log.

7. Player Performance & Leaderboards

Recommended player dashboard

Metric	Purpose
Matches played	Activity
Wins / podiums	Competitive outcomes
Kills / assists / objectives	Game-specific performance
Average placement	Consistency
Points per match	Scoring efficiency
Win rate	Outcome metric
Recent form	Recent performance without replacing long-term stats
Tournament history	Evidence of experience
Team history	Role and team participation
Fair-play record	Confirmed penalties/reports, shown according to policy

Use separate leaderboards for different games and seasons. A Free Fire score should not be directly compared with a different game's score. If you create a platform-wide ranking, define a normalisation formula and publish it.

Leaderboard update flow

Match result submitted -> validation -> organiser confirmation / automated verification -> result finalisation -> scoring engine -> leaderboard transaction -> cached leaderboard refresh -> real-time event -> push notification. Keep a historical snapshot so rankings can be reproduced later.

8. Internal Currency & Redeem-Code System

A safer technical model is to treat the platform currency as a closed-loop internal credit, not as a general-purpose cash balance. The database should use a double-entry-style ledger so that no administrator directly overwrites a user's balance.

> **Important:** whether users may purchase credits and whether credits may be converted back into money/prizes can create financial, consumer-protection, tax, gaming, payments, KYC/AML and state-specific legal obligations. The production version should be reviewed by qualified legal/payment professionals before launch.

Redeem code lifecycle

- Admin selects approved credit denomination and quantity.
- Backend generates cryptographically random, high-entropy codes; never use sequential codes.
- Store only a secure hash of the redeem code where practical; show the plaintext code only at controlled creation/export time.
- Code states: CREATED -> ACTIVE -> REDEEMED / EXPIRED / CANCELLED / BLOCKED.
- Each code has value, creation batch, expiry, max uses (normally 1 for monetary value), creator, timestamp and redemption user.
- Rate-limit redemption and flag unusual attempts.
- Never allow admin UI to silently edit an already redeemed transaction.
- Vexora addition: keyed hashing (HMAC with a secret from the secret manager) plus a checksum character so typos are rejected before a database lookup.
- Vexora addition: batch export is encrypted, watermarked with the exporting admin ID, and the export itself is audit-logged.

Credit ledger

Example ledger structure: transaction_id | user_id | type | credit_amount | reference_type | reference_id | status | created_at | created_by | idempotency_key

- Credits purchased/redeemed -> CREDIT transaction.
- Tournament reservation -> HOLD transaction.
- Tournament cancelled -> RELEASE/REFUND transaction.
- Approved reward/prize -> REWARD transaction where permitted.
- Approved withdrawal/settlement -> DEBIT/SETTLEMENT transaction.
- Correction -> ADJUSTMENT with mandatory reason and admin identity.

Sell-back / payout request flow

If the business model legally permits cash settlement, do not make it an unrestricted peer-to-peer cash-out system. Use a controlled request flow: user submits eligible amount + required payment/KYC details -> compliance checks -> risk checks -> finance review -> payment-provider settlement -> webhook confirmation -> ledger finalisation. Never mark a payout successful merely because an admin clicked a button; reconcile it against the payment provider/bank record.

9. Payments, Prize Pools & Financial Controls

- Keep tournament entry, platform credits, prizes and organiser fees as separate accounting concepts.
- Display exactly what a player pays/uses and what they may receive according to the tournament terms.
- Publish cancellation/refund rules before registration.
- Use payment-provider webhooks for payment confirmation rather than trusting the client.
- Use idempotency keys for payment creation and settlement.
- Daily reconciliation: internal ledger vs payment gateway vs payout records.
- Maintain tax/invoice records as required by the applicable jurisdiction.
- Add a withdrawal/payout review queue with statuses: requested, under review, approved, processing, paid, failed, rejected.
- Vexora addition: prize escrow — organiser-funded prize pools are locked in a dedicated escrow account before registration opens.
- Vexora addition: verify webhook signatures and replay-protect them with timestamp + event-ID de-duplication.

10. Organiser Verification & Trust

Application information

- Full legal name and contact information.
- Age/eligibility confirmation as required by the service.
- Identity verification details where legally required.
- Payment/settlement details where required.
- Gaming/community experience and prior tournament history.
- Rules acceptance and organiser code of conduct.
- Emergency/support contact and dispute-handling commitment.

Do not expose sensitive identity documents or exact location to ordinary users. Store sensitive documents separately with strong access controls. An organiser's public profile should show verification status, hosting history, completion statistics, participant feedback and confirmed policy violations—not private KYC data.

Trust score

Instead of a vague "trusted" label, show transparent indicators such as number of completed tournaments, cancellation rate, unresolved disputes and verification status. Avoid using user reports alone as proof of wrongdoing.

> **Vexora addition.** Introduce organiser tiers: Starter (free tournaments only, capped capacity), Verified (paid-entry allowed after review), and Partner (higher caps, priority support). Promotion between tiers is rule-based and visible, and demotion/hold triggers (e.g. high dispute rate) are documented.

11. Anti-Cheat, Anti-Fraud & Dispute System

Principles

- Use evidence-based moderation.
- Keep an audit trail for every moderation decision.
- Separate a report from a confirmed violation.
- Give users a clear dispute/appeal route.
- Use automated risk signals to prioritise reviews, not to automatically punish users in high-impact cases.
- Preserve relevant screenshots, result screens, chat records and event logs according to a defined retention policy.

Useful risk signals

Signal	Possible action
Many accounts using same device/payment pattern	Risk review
Repeated suspicious result corrections	Moderator review
High dispute rate from one organiser	Temporary tournament publishing hold
Unusual redeem-code attempts	Rate-limit + security review
Repeated charge/payment failures	Payment review
Impossible score/time combination	Result validation
Rapid account creation	Verification/risk controls

Location: do not design the system around secretly tracking or physically punishing users. If approximate location is genuinely necessary for security or regional eligibility, collect it transparently, minimise precision, secure it, define retention, and comply with applicable privacy law. The platform should never encourage users or organisers to confront or harm someone.

Dispute flow

Report created -> evidence attached -> case ID -> temporary protection/hold if necessary -> moderator review -> both sides notified -> decision with reason -> financial/result correction if authorised -> appeal window -> final decision -> audit log. During a dispute, freeze the affected settlement rather than deleting records.

12. Admin Desktop Dashboard

Dashboard module	Key controls
Overview	Active users, live tournaments, pending organisers, disputes, payout queue, wallet liability
Users	Search, profile, verification, status, team history, tournament history, reports
Organisers	Applications, verification, performance, suspension, tournament review
Games	Games, modes, scoring templates, rule versions
Tournaments	Search, approvals, live status, brackets, results, disputes
Wallet	Ledger search, code batches, reconciliation, adjustments with reason
Payouts	Requests, verification, approval, failure/retry, audit
Moderation	Reports, evidence, cases, sanctions, appeals
Notifications	Push/in-app/email templates and targeted announcements
Analytics	Users, retention, tournament completion, disputes, revenue/settlement metrics
System	Roles, permissions, feature flags, maintenance, audit logs
Security Centre (Vexora addition)	Live threat feed, blocked IPs, failed-login map, admin session list, break-glass requests, integrity-check results

13. Mobile Application

Player navigation

Home -> Games -> Tournaments -> My Matches -> Teams -> Leaderboard -> Wallet -> Notifications -> Profile -> Help/Disputes

Organiser navigation

Dashboard -> My Tournaments -> Create -> Participants -> Matches -> Results -> Evidence -> Announcements -> Analytics -> Profile/Verification -> Support

Important mobile features

- Fast tournament filters.
- Real-time match status.
- Push notifications for registration, check-in, lobby release, result confirmation and disputes.
- QR/deep-link support for joining a tournament or opening a match.
- Offline-safe form drafts for organiser match operations, followed by server reconciliation.
- Secure session/token handling.
- App version enforcement for critical security updates.
- Vexora addition: biometric unlock (fingerprint/face) for wallet and payout screens, using the OS secure keystore.
- Vexora addition: root/jailbreak and emulator signals feed the risk engine (do not hard-block; raise risk score).
- Vexora addition: Play Integrity (Android) / App Attest (iOS) checks on sensitive API calls.
- Vexora addition: screenshot-protected screens for redeem codes and room IDs (FLAG_SECURE on Android).
- Vexora addition: low-data mode, dark mode, multi-language (English, Hindi, Odia and other regional languages), accessibility labels.

14. Real-Time Architecture

Use a conventional REST/HTTP API for most CRUD operations and WebSockets (or a managed real-time service) for events that need immediate updates.

Event	Who receives it
Tournament registration	Organiser/admin
Tournament status change	Registered players/organiser
Match starts	Relevant teams/players
Result submitted	Opponent/organiser/moderator as configured
Result finalised	Participants + leaderboard subscribers
Dispute opened/updated	Affected parties + moderators
Wallet transaction	User + finance/admin where applicable
Payout status	User + finance
Announcement	Target audience

Vexora addition. Authenticate every WebSocket connection with a short-lived token, authorise each channel subscription server-side (a player can only join channels of tournaments they are registered in), apply per-connection message rate limits, and use Redis pub/sub so multiple backend instances can fan out events. Add heartbeat + automatic reconnect with missed-event replay from a sequence number.

15. Recommended Technical Architecture

Option A — practical modern stack

Layer	Recommended technology
Mobile	Flutter or React Native (Vexora default: Flutter, built with the Dart/Flutter MCP server)
Web	React / Next.js
Admin	Next.js/React with strict RBAC
Backend	Node.js + NestJS/Express or Java + Spring Boot
Database	PostgreSQL
Cache / queues	Redis
Real-time	WebSockets / Socket.IO or managed real-time service
Storage	S3-compatible object storage for evidence/images
Auth	OAuth/phone/email + JWT/session rotation; MFA for admins
Search	PostgreSQL initially; OpenSearch/Elastic later if required
Notifications	Firebase Cloud Messaging + email/SMS provider
Monitoring	Sentry + metrics/logging stack
Deployment	Docker + managed cloud services; CI/CD

For a student/startup MVP, do not begin with microservices. Use a modular monolith with clear modules and background workers. Split services only when scale or team structure justifies it.

High-level architecture

Mobile Apps / Web -> API Gateway / Backend -> Auth & RBAC -> Tournament Module -> Match/Bracket Engine -> Scoring/Leaderboard Engine -> Wallet/Ledger -> Payment/Code Module -> Notification Service -> PostgreSQL + Redis + Object Storage -> Admin/Moderation Console

16. Database Design

Table	Important fields
users	id, username, phone/email, status, created_at, last_login
user_roles	user_id, role_id
games	id, name, icon, status
game_modes	id, game_id, mode, team_size, rules_version
teams	id, name, tag, owner_id, game_id, status
team_members	team_id, user_id, role, joined_at, status
organiser_profiles	user_id, verification_status, public_stats
organiser_documents	organiser_id, type, secure_ref, review_status
tournaments	id, organiser_id, game_id, mode_id, status, schedule, capacity
tournament_rules	tournament_id, version, rule_json, published_at
registrations	tournament_id, team_id/user_id, status, created_at
matches	id, tournament_id, round, participants, status, scheduled_at
match_results	match_id, submitted_by, result_data, evidence_ref, status
scores	match_id, participant_id, points, kills, placement, metadata

Table	Important fields
leaderboard_entries	season_id, game_id, participant_id, points, rank
seasons	id, game_id, name, start_at, end_at
wallet_accounts	user_id, currency, available_balance, held_balance
ledger_transactions	id, user_id, type, amount, ref, status, idempotency_key
redeem_codes	id/hash, value, batch_id, status, expires_at, redeemed_by
payout_requests	id, user_id, amount, status, payment_ref
reports	id, reporter_id, target_type, target_id, category, status
disputes	id, tournament_id/match_id, opened_by, status, decision
evidence	id, case_id, object_storage_ref, hash, uploaded_by
notifications	id, user_id, type, payload, read_at
audit_logs	id, actor_id, action, entity, before, after, timestamp

Vexora addition: extra tables

Table	Important fields
user_devices	id, user_id, device_fingerprint_hash, platform, push_token, first_seen, last_seen, trust_level
user_sessions	id, user_id, device_id, refresh_token_hash, created_at, expires_at, revoked_at, ip_hash
mfa_factors	id, user_id, type (totp/webauthn/sms), secret_ref, verified_at
feature_flags	key, enabled, rollout_percent, audience, updated_by
support_tickets / ticket_events	id, user_id, category, priority, sla_due_at, owner_id; event history
kyc_cases	id, user_id, provider_ref, status, reviewed_by, reviewed_at (documents stored separately and encrypted)
achievements / user_achievements	badge definitions; user_id, badge_id, awarded_at, source_ref
audit_chain_checkpoints	id, from_log_id, to_log_id, merkle_root, signed_at (tamper-evidence)
webhook_events	id, provider, event_id (unique), signature_ok, payload_hash, processed_at
risk_events	id, user_id, signal, score, context_json, created_at, reviewed_by

Vexora addition. Schema rules: UUIDv7 primary keys, NUMERIC (never float) for money/credits, CHECK constraints so balances cannot go negative, UNIQUE(idempotency_key), append-only ledger enforced with a trigger that forbids UPDATE/DELETE, timestamps in UTC (timestamptz), soft-delete only where retention policy requires, and all migrations versioned and reviewed.

17. API Structure

Example REST groups:

```
/auth/*   /users/*   /games/*   /teams/*   /organisers/*   /tournaments/*
/matches/*   /results/*   /leaderboards/*   /wallet/*   /redeem/*   /payouts/*
/reports/*   /disputes/*   /notifications/*   /admin/*
(Vexora addition) /support/*   /achievements/*   /devices/*   /security/*   /webhooks/*   /health
```

Secure endpoints server-side. Never trust client-sent role, balance, prize amount, tournament status or result score. The backend must re-check authorisation and business rules for every sensitive request.

Vexora addition. Version the API (/v1), publish an OpenAPI spec generated from code, validate every request body with a schema (zod / class-validator), use cursor pagination, return consistent error codes, and require an Idempotency-Key header on all money/result-changing POST requests.

18. Security Requirements

- HTTPS everywhere; HSTS for web.
- Secure password hashing (Argon2id/bcrypt) if passwords are used.
- Short-lived access tokens + refresh-token rotation or secure server sessions.
- MFA for administrators and high-risk finance/moderation actions.
- RBAC + least privilege.
- Rate limiting on login, OTP, redeem-code, report and payout endpoints.
- Server-side validation and output encoding.
- CSRF protection for cookie-based sessions.
- SQL injection protection through parameterised queries/ORM.
- Object-storage access through short-lived signed URLs.
- Encryption at rest for sensitive records where appropriate.
- Secrets stored in a secret manager, never in source code.
- Immutable/append-only audit logging for high-impact operations.
- Backups with restore testing.
- Dependency and container vulnerability scanning.
- Device/session management and suspicious-login alerts.
- Privacy deletion/retention workflow consistent with legal requirements.

Vexora additions to the baseline above are detailed in Section 47 (Extended Security Architecture).

19. Anti-Abuse Controls

Threat	Control
Fake accounts	Phone/email verification, device/risk signals, rate limits
Account takeover	MFA, new-device alerts, session management
Redeem-code theft	High-entropy codes, one-time use, hashed storage, rate limits
Admin abuse	RBAC, MFA, dual approval, audit logs
Result manipulation	Evidence, immutable submissions, validation, disputes
Organiser fraud	Verification, history, holds, dispute process
Payment fraud	Provider webhooks, reconciliation, velocity limits
Spam/abuse	Moderation, report limits, chat controls
Data leakage	Least privilege, signed URLs, encryption, access logs
Bots / scripted registration (Vexora)	CAPTCHA/Turnstile on risky flows, proof-of-work on bursts, device attestation
Collusion / soft-play (Vexora)	Statistical anomaly review of repeated opponent pairings and unusual placement patterns
Smurf / multi-account (Vexora)	In-game UID uniqueness, device clusters, review queue (never auto-ban)

20. Notifications & Communication

- Push: tournament reminders, check-in, match start, result updates, dispute updates, payout status.
- In-app inbox for durable notifications.
- Email for important account/security events.
- Optional SMS/OTP for verification where legally and operationally appropriate.
- Tournament announcement channel controlled by organiser with moderation.
- Do not expose private contact information between participants.
- Vexora addition: user notification preferences (per category, quiet hours), and mandatory non-mutable security alerts.
- Vexora addition: templated, localised messages with delivery tracking and retry/dead-letter queue.

21. Ratings & Reputation

After a completed tournament, allow participants to rate the organiser on clearly defined operational factors such as communication, schedule adherence and result handling. Also allow organisers to report participant behaviour. Ratings should not be the sole basis for punishment.

- Show sample size.
- Separate verified completed tournaments from unverified comments.
- Allow a dispute/appeal for demonstrably false or abusive content.
- Use moderation for harassment, personal information and threats.
- Do not allow organisers to delete negative reviews themselves.

22. Analytics

Player analytics

- Performance by game and season.
- Trend of points, placements and relevant game statistics.
- Team contribution and role history.
- Match consistency and recent form.
- Tournament participation and completion.
- Fair-play/dispute history shown only according to published privacy policy.

Organiser analytics

- Registrations per tournament.
- Check-in rate.
- Completion rate.
- Average dispute rate.
- Average result-finalisation time.
- Participant rating trends.
- Cancellation/refund rate.

Admin analytics

- DAU/MAU, retention and acquisition.
- Tournament supply/demand.
- Wallet liability and settlement reconciliation.
- Payout processing time.

- Fraud/risk cases.
- Support workload.
- Platform uptime and API latency.

23. Support System

Create a ticket system rather than relying only on WhatsApp/Instagram. Each issue gets a ticket ID, category, priority, SLA, owner and audit history.

Category	Examples
Account	Login, verification, account recovery
Tournament	Registration, schedule, rules
Match	Result, evidence, lobby issue
Payment	Payment pending, failed, duplicate
Wallet	Redeem, balance/ledger question
Payout	Pending, failed, rejected
Safety	Harassment, threats, suspicious activity
Organiser	Verification, publishing, dispute

24. Legal, Compliance & Policy Layer

Before production launch, obtain jurisdiction-specific advice. The exact treatment of esports entry fees, prize pools, platform credits, cash settlement, taxes, KYC/AML, consumer protection, advertising, age restrictions, payment processing and gaming laws can differ by location and can change over time.

- Terms of Service.
- Privacy Policy and data-retention policy.
- Refund/cancellation policy.
- Tournament rules and organiser agreement.
- Code of conduct.
- Dispute and appeal policy.
- Payout/settlement terms.
- Acceptable use and anti-fraud policy.
- Age/eligibility rules.
- Consent and privacy controls for location/device data.
- Tax/invoice and accounting process.
- Clear statement that the platform does not support cheating, match-fixing or wagering unless separately lawful and expressly supported.
- Vexora addition: India-specific review items — DPDP Act 2023 consent and data-principal rights, IT Rules intermediary obligations (grievance officer, takedown timelines), state-level online-gaming rules, GST treatment of entry fees/credits, RBI/payment-aggregator rules, and minors/parental-consent policy. Confirm each with a qualified Indian lawyer.

25. Trust Features That Make Users Comfortable

- Verified organiser badge with verification date/status.
- Public organiser history: completed, cancelled and disputed tournaments.
- Visible tournament rules before joining.
- Prize distribution shown before registration.
- Published cancellation/refund conditions.
- Match/result evidence and dispute workflow.
- Wallet transaction history that users can inspect.
- Unique transaction IDs and support ticket IDs.
- Real-time tournament status.
- Clear contact/support channel.
- No silent balance edits or silent result edits.
- Admin actions recorded in audit logs.
- Security page explaining account protection and responsible reporting.
- Vexora addition: "Verify this result" page showing result hash, evidence hashes, timestamps and the scoring formula version used.
- Vexora addition: public transparency report (disputes resolved, average resolution time, takedowns) published periodically.

26. Important Edge Cases

- Tournament does not reach minimum participants -> apply published cancellation/refund rule.
- Organiser cancels -> lock settlement and start refund/dispute workflow.
- Player disconnects -> use game-specific published rule; do not improvise after the match.
- Two teams submit conflicting results -> hold result and open dispute.
- Evidence is missing -> result remains pending until rule-defined timeout/action.
- Payment succeeds but app did not update -> reconcile provider webhook and idempotency key.
- Redeem code is entered twice -> second request returns already-used status without changing balance.
- Payout fails -> return to pending/failed state without losing ledger history.
- Admin makes a mistake -> correction transaction, never destructive database editing.
- User deletes account -> retain legally required financial/audit records while removing/anonymising other data as policy permits.
- Server outage during live tournament -> preserve match state and provide status/maintenance communication.
- App version is outdated -> block only functions requiring the newer security protocol.
- Vexora addition: organiser goes offline mid-tournament -> auto-escalate to moderator after a published timeout; allow co-organiser takeover.
- Vexora addition: two simultaneous registrations for the last slot -> row-level lock / unique constraint; loser is waitlisted and not charged.
- Vexora addition: device clock is wrong -> all timing decisions use server time only.
- Vexora addition: evidence file is corrupted or malware-flagged -> quarantine, notify uploader, keep original hash in the log.

27. Development Roadmap

Phase	Build	Output
0. Planning	Requirements, legal review, UX, data model	Approved specification
1. Foundation	Auth, users, roles, database, admin shell	Secure account system
2. Games & Teams	Games, modes, teams, profiles	Player ecosystem
3. Tournament MVP	Create, discover, register, brackets, match status	Working tournament engine
4. Results	Result submission, evidence, scoring, leaderboard	Competitive workflow
5. Trust	Organiser verification, reports, disputes, moderation	Safety layer
6. Wallet	Ledger, redeem codes, payment integration	Controlled credit system
7. Real-time	WebSockets, push, live status	Live experience
8. Analytics	Player/organiser/admin dashboards	Performance intelligence
9. Hardening	Security testing, load testing, backup/DR, app-store prep	Release candidate
10. Launch	Pilot with limited organisers/games	Controlled production launch

Vexora addition. Section 39 expands every phase above into goals, tasks, the exact MCP servers to use, a security gate and exit criteria, so each phase can be built, tested and signed off before the next one starts.

28. MVP Scope vs Later Features

MVP	Later
Login + profile	Advanced social login
Game selection	Large game catalogue
Teams	Advanced team contracts/management
Tournament create/join	Advanced Swiss/custom formats
Basic bracket	Dynamic tournament formats
Result submission	Automated game API result ingestion where officially supported
Leaderboard	Advanced ranking/normalisation
Organiser verification	Automated verification workflows
Basic wallet ledger	Expanded payment/settlement options after compliance review
Redeem codes	Promotional campaigns and advanced code rules
Reports/disputes	Advanced risk scoring
Push notifications	Personalised recommendation engine

Vexora recommendation. Launch the first public version with FREE tournaments only (no entry fee, no cash prize), proving the competition engine, trust layer and organiser quality. Enable credits and rewards in a later, legally reviewed release.

29. Testing Plan

- Unit tests for scoring, wallet ledger, tournament state transitions and permissions.
- Integration tests for registration -> payment/credit hold -> match -> result -> settlement.
- Security tests for authentication, authorisation, injection, rate limits and file uploads.
- Concurrency tests: two users registering for the last slot simultaneously.
- Idempotency tests: duplicate payment/result/webhook requests.
- Load tests for tournament start spikes and leaderboard refreshes.
- Mobile tests across low-end Android devices and unreliable networks.
- Disaster recovery test: restore database and object-storage references from backup.
- Manual abuse testing for organiser/player cheating scenarios.
- App-store policy and privacy review before publication.

Vexora additions

- Playwright (via MCP) end-to-end suites for the web and admin apps: login, MFA, organiser application, tournament publish, registration, result dispute, redeem code, payout approval.
- Flutter widget, golden and integration tests run through the Dart MCP server.
- Property-based tests for the ledger (sum of all entries is always zero; balance never negative).
- Contract tests between the mobile app and the OpenAPI spec.
- Mutation testing on the scoring engine; fuzzing on file-upload and result parsers.
- Accessibility tests (screen reader, contrast, large fonts) and localisation tests.

30. Suggested Folder Structure

```
vexora/
  apps/
    mobile/            (Flutter: player + organiser flavours)
    web/               (Next.js: public site + organiser web)
    admin/             (Next.js: admin console, strict RBAC)
  backend/
    src/
      auth/  users/  games/  teams/  organisers/  tournaments/  matches/
      scoring/  leaderboards/  wallet/  redeem/  payouts/  disputes/
      moderation/  notifications/  admin/  audit/
      support/  achievements/  risk/  webhooks/      (Vexora additions)
  packages/
    shared-types/  validation/  scoring-engine/
  infra/
    docker/  migrations/  ci/
    terraform/  seeds/                               (Vexora additions)
  docs/
    api/  rules/  policies/
    adr/  runbooks/  threat-models/  mcp/            (Vexora additions)
  .mcp.json            (MCP server config, no secrets)
  CLAUDE.md            (project rules for the AI assistant)
  .env.example         (placeholders only)
```

31. Core Business Rules to Lock Before Coding

- What exactly counts as a tournament entry and what is free vs paid?
- Which games are supported at launch?
- Which tournament formats are supported?
- Who funds prizes and when is the prize considered locked?
- What happens if the minimum participant count is not reached?
- What is the cancellation/refund policy?
- When is a result final?
- How long can a dispute be opened?
- Who can change an official result?
- What organiser verification is mandatory?
- What is the internal credit model and what legal permissions are required for every conversion?
- What user information is mandatory, and what is optional?
- What data is public on a player/team/organiser profile?
- What sanctions exist and what is the appeal process?
- What are the exact scoring formulas for every game?
- Vexora addition: minimum player age and the parental-consent rule for minors.
- Vexora addition: which Indian states/countries are enabled at launch (geo-eligibility list owned by legal).
- Vexora addition: data-retention period for each data class (evidence, chat, KYC, logs).
- Vexora addition: who is the grievance officer and what are the response timelines?

> **Rule:** record every answer above as an Architecture Decision Record (ADR) in docs/adr before the related code is written.

32. Example End-to-End Tournament

Example: An approved organiser creates a 16-team tournament. The organiser selects the game/mode, publishes the schedule, rules and prize structure, and submits the tournament for approval. Once published, teams register. The system validates capacity, eligibility and any required credits. At check-in, the roster is locked. Matches are created by the bracket engine. After each match, the organiser submits the result and evidence. The backend validates the submission, the opposing team may confirm or dispute it, and a moderator can review conflicts. Once final, the scoring engine updates the tournament leaderboard and the player's season statistics. When the tournament reaches the final state, settlement follows the published rules and any applicable financial/compliance checks. All high-impact actions remain traceable by transaction IDs and audit logs.

33. What Makes This More Than a Tournament App

- It becomes a player performance identity: a structured history of skill, consistency, teamwork and fair play.
- It becomes an organiser marketplace only after strong verification and trust controls are in place.
- It becomes a competition infrastructure layer: game -> rules -> tournament -> match -> result -> score -> ranking.
- It creates a transparent internal financial/accounting trail rather than relying on manually edited balances.
- It can eventually support seasons, badges, achievements, verified statistics, team scouting and creator/organiser tools—subject to game publisher rules and applicable law.

34. Final Recommended Product Architecture

Component	Summary
PLAYER	Mobile-first app for discovery, teams, tournament participation, results, wallet history, rankings and performance.
ORGANISER	Mobile + web workspace for verified tournament creation, participant management, match operations, result/evidence submission and analytics.
ADMIN	Desktop-first secure console for verification, tournament oversight, wallet/code management, payouts, moderation, disputes, games/rules, analytics and audit logs.
BACKEND	Modular API + PostgreSQL + Redis + object storage + WebSockets + push notifications + background jobs.
TRUST	Verification + published rules + evidence + dispute process + ledger + audit trail + security controls.
REAL-TIME	Tournament/match/leaderboard events delivered through WebSockets and push notifications.
IDENTITY & PUSH (Vexora)	Firebase Authentication (phone OTP/email/Google), Firebase Cloud Messaging, App Check, Crashlytics, Remote Config.
SYSTEM OF RECORD (Vexora)	PostgreSQL holds all money, tournament, result and audit data. Firebase never stores balances or results.

35. Launch Checklist

- [ ] Product name, domain and branding selected (name: Vexora; verify domain, trademark and app-store name availability)
- [ ] Supported games and publisher/API permissions checked
- [ ] Tournament rules templates completed
- [ ] Organiser verification policy completed
- [ ] Terms, privacy, refund and dispute policies reviewed by qualified professionals
- [ ] Payment/credit/prize model legally reviewed for target jurisdictions
- [ ] Database schema and ledger design reviewed
- [ ] RBAC and admin MFA implemented
- [ ] Tournament state machine implemented
- [ ] Scoring formulas versioned
- [ ] Evidence storage secured
- [ ] Audit logging enabled
- [ ] Backup and disaster recovery tested
- [ ] Security testing completed
- [ ] Load testing completed
- [ ] Pilot organisers verified
- [ ] Pilot tournament completed end-to-end
- [ ] Support process ready
- [ ] App privacy disclosures and store requirements completed
- [ ] Vexora: MCP servers configured with least privilege and documented (Section 38)
- [ ] Vexora: penetration test findings (critical/high) closed
- [ ] Vexora: incident-response runbook rehearsed once
- [ ] Vexora: status page and in-app maintenance banner ready
- [ ] Vexora: Firebase App Check enforced on all production apps

36. Conclusion

The strongest version of this idea is not simply an app where players pay to enter tournaments. It is a trusted esports competition platform that connects verified organisers, teams and players, records competitive performance, provides transparent tournament rules and results, and maintains a traceable ledger for platform credits and settlements.

Build the competition engine and trust layer first. Add real-money functionality only after the legal/payment model is reviewed and the internal ledger, dispute handling, reconciliation and security controls are production-ready. Start with a small number of games and a controlled organiser pilot, then expand based on measured tournament completion, dispute rates, retention and operational reliability.

PART II

Vexora additions: MCP toolchain, phase-by-phase build plan, extended features and hardened security

37. MCP Toolchain for Building Vexora

MCP (Model Context Protocol) servers give the AI coding assistant safe, structured access to tools and live documentation, so it can read current docs, run analysers and tests, drive a browser, query a database and manage the repository instead of guessing. Vexora uses nine MCP servers. Each has one clear job and is introduced in the phase where it first matters.

MCP server	What it does for Vexora	Main phases
Dart MCP server	Official Dart/Flutter tooling for the assistant: analyse code, fix lints, run tests, format, inspect the running app/widget tree, search pub.dev and manage dependencies. Builds the Android/iOS apps.	0, 1, 2, 3, 4, 7, 9
Context7	Fetches up-to-date, version-specific documentation and code examples for libraries (NestJS, Next.js, Flutter packages, Socket.IO, Prisma/Drizzle, Razorpay SDK and so on) so generated code does not use outdated APIs.	All phases
Filesystem	Scoped read/write access to the Vexora project folder: create modules, edit files, organise the monorepo, generate docs and migrations.	All phases
Stitch	AI UI design tool: generates screen designs and front-end code from prompts. Used for the Vexora design system, player/organiser app screens and the admin dashboard layouts before they are built in Flutter/Next.js.	0, 1, 2, 3, 8
Developer Knowledge	Search of Google's official developer documentation (Firebase, Android, Google Cloud, Flutter-adjacent guidance) so Firebase Auth, FCM, App Check and Play Integrity are implemented per the current official guidance.	1, 7, 9, 10
Firebase	Manage the Firebase project from the assistant: Auth providers, Cloud Messaging, Remote Config, App Check, Crashlytics, Analytics, security rules and app registration.	1, 6, 7, 9, 10
Playwright	Browser automation: end-to-end tests for the web, organiser and admin apps, regression suites, accessibility checks and screenshot comparison.	1, 3, 4, 5, 6, 9
GitHub	Repository work: branches, issues, pull requests, code review, Actions status, code-scanning/Dependabot alerts and release notes. Drives the phase-by-phase milestone board.	All phases
Postgres	Inspect the development/staging database: schema, indexes, query plans and test data. Used read-only to verify migrations, the ledger invariants and slow queries.	1, 3, 4, 6, 8, 9

How the servers work together

```
Stitch (design) -> Dart MCP (Flutter UI) + Filesystem (code)
   -> Context7 / Developer Knowledge (correct, current APIs)
   -> Postgres (verify schema/data) -> Playwright (E2E tests)
   -> GitHub (PR, review, CI) -> Firebase (auth, push, release)
```

Division of responsibility. PostgreSQL is the system of record for money, tournaments, results and audit logs. Firebase handles identity (phone/email OTP), push notifications, App Check, Remote Config and crash reporting. The backend verifies Firebase ID tokens and maps them to Vexora users. Balances and results are never stored in Firebase.

38. MCP Configuration & Safe-Use Rules

Example project-level configuration (.mcp.json). Package names and flags change over time, so confirm each against the server's current documentation before use. Secrets are read from environment variables and never committed.

```
{
  "mcpServers": {
    "dart":       { "command": "dart", "args": ["mcp-server"] },
    "context7":   { "command": "npx",  "args": ["-y", "@upstash/context7-mcp"] },
    "filesystem": { "command": "npx",  "args": ["-y", "@modelcontextprotocol/server-filesystem",
                                                 "./vexora"] },
    "firebase":   { "command": "npx",  "args": ["-y", "firebase-tools@latest", "mcp"] },
    "playwright": { "command": "npx",  "args": ["@playwright/mcp@latest"] },
    "github":     { "type": "http", "url": "<GitHub MCP endpoint>",
                   "headers": { "Authorization": "Bearer ${GITHUB_MCP_TOKEN}" } },
    "postgres":   { "command": "npx",  "args": ["-y", "<postgres-mcp-package>"],
                   "env": { "DATABASE_URL": "${VEXORA_DEV_RO_DATABASE_URL}" } },
    "stitch":     { "...": "per the Stitch MCP setup guide" },
    "developer-knowledge": { "...": "per Google Developer Knowledge MCP setup guide" }
  }
}
```

Per-server safe-use rules

Server	Rule
Filesystem	Allow only the Vexora project directory. Never expose home, SSH keys, cloud credentials or other projects.
Postgres	Connect with a dedicated READ-ONLY role to the DEV or STAGING database only. Never give it production credentials. Prefer a maintained server that enforces read-only mode (the original reference Postgres server is archived, so check the current recommended option).
GitHub	Use a fine-grained token limited to the Vexora repository (contents, issues, pull requests, actions read). No admin or org-wide scopes. Branch protection stays on so the assistant cannot push to main.
Firebase	Point at the DEV project during development; production project changes go through a reviewed release step. Restrict the account to the roles needed.
Playwright	Run against local/staging URLs with test accounts only. Never log into real accounts, never store real credentials in test files.
Dart / Context7 / Stitch / Developer Knowledge	Low risk (read/analyse/design). Still pin versions and review generated code.

Golden rules for AI-assisted development

- Least privilege: every MCP server gets the minimum scope; separate dev, staging and production credentials.
- Human approval for anything destructive or outward-facing: migrations on shared databases, force pushes, deleting data, publishing releases, changing Firebase production settings.
- Treat external content as untrusted: text in GitHub issues, web pages, user-submitted data or documentation can contain instructions aimed at the assistant (prompt injection). Do not let such text trigger actions without review.
- Never paste secrets into prompts. Use environment variables or a secret manager; keep .env in .gitignore and run secret-scanning in CI.
- All AI-generated code goes through the same path as human code: pull request, automated tests, security scan, human review.
- Pin MCP package versions in the lock file and review changelogs before upgrading; remove servers you no longer use.
- Keep a CLAUDE.md in the repository containing coding standards, the security rules above, the module map and the current phase so every session starts with the same context.
- Log which MCP tools were used in each pull request description so reviews can focus on risky operations.

39. Phase-by-Phase Build Plan

Vexora is built in eleven phases (0-10). Each phase is a vertical slice that ends with working, tested, deployed software. Estimates assume a small team (1-3 developers) working with MCP-assisted tooling; adjust to your team size. Every phase follows the same loop:

```
1. Plan  : GitHub milestone + issues, update CLAUDE.md with the phase goal
2. Design: Stitch screens + API/DB design (ADR for any big decision)
3. Build : small branches -> pull requests (Dart MCP, Filesystem, Context7, Postgres)
4. Test  : unit + integration + Playwright E2E + security checks in CI
5. Review: code review, security gate checklist, demo on staging
6. Ship  : merge, tag release, update docs and the changelog; retrospective
```

Phase	Name	Est.	Headline result
0	Planning, setup & design	2-3 wks	Approved spec, repo, MCP toolchain, design system
1	Foundation: auth, users, roles, admin shell	3 wks	Secure account system
2	Games & teams	3 wks	Player ecosystem
3	Tournament MVP	4 wks	Working tournament engine
4	Results, evidence, scoring, leaderboards	3 wks	Competitive workflow
5	Trust: verification, reports, disputes, moderation	3 wks	Safety layer
6	Wallet, ledger, redeem codes, payments	4 wks	Controlled credit system
7	Real-time, push & live experience	2 wks	Live tournaments
8	Analytics, achievements & support desk	3 wks	Performance intelligence
9	Hardening: security, load, DR, store prep	3 wks	Release candidate
10	Pilot & controlled launch	3 wks+	Production launch

> **Timeline.** Total is roughly 32-36 weeks for a small team. Phases can overlap slightly (for example Stitch designs for Phase N+1 during Phase N), but a phase is not "done" until its exit criteria are met.

PHASE 0 — Planning, Setup & Design — Est. 2-3 weeks

Goal. Turn this blueprint into locked decisions, a clean repository, a working MCP toolchain and a Vexora design system.

What to build

- Answer every question in Section 31 and record decisions as ADRs; get the legal review started (Section 24).
- Create the GitHub repository (monorepo), branch protection, issue templates, PR template, CODEOWNERS and the milestone board with one milestone per phase.
- Scaffold the folder structure from Section 30; add CLAUDE.md, .mcp.json, .env.example, .gitignore and pre-commit hooks (formatting, secret scan).
- Set up local development with Docker Compose: PostgreSQL, Redis, MinIO (S3-compatible), mail catcher.
- Create the Firebase DEV and PROD projects (Auth, FCM, Crashlytics, Remote Config, App Check); register Android and iOS apps.
- Create the Vexora brand and design system: logo, colours, typography, components, dark theme, motion rules (Section 44).
- Draw the data model (ERD) and the first OpenAPI draft; write the threat model for auth, wallet and results.
- Stand up CI (lint, test, build, dependency and secret scanning) even if tests are still empty.

MCP servers used in this phase

MCP server	How it is used
Filesystem	Scaffold the monorepo, ADRs, docs and config files.
GitHub	Create repository settings, milestones, issues and the PR template; confirm branch protection.
Context7	Check current setup guides for NestJS, Next.js, Flutter packages and the ORM before choosing versions.
Stitch	Generate first design directions for the Vexora brand, home, tournament card and admin overview.
Developer Knowledge	Read current Firebase project setup and Play Integrity/App Check guidance.
Firebase	Create/configure the dev project, register apps, enable phone and email authentication.
Dart MCP	Create the Flutter project, set lint rules, run first analysis and a smoke test.
Postgres	Connect read-only to the local dev database to confirm the connection and extensions.
Playwright	Install and run a smoke test against the placeholder web app.

Security gate

- MCP servers use least-privilege credentials (Section 38); no secrets in the repository.
- Branch protection: PR required, status checks required, no force-push to main.
- Threat model v1 written (auth, wallet, results, admin).

Exit criteria (do not start the next phase until all are true)

[ ] All Section 31 questions answered and recorded as ADRs.

[ ] Repo, CI, Docker environment and MCP toolchain working for every team member.

[ ] Design system v1 approved; legal review engaged.

[ ] Phase 1 issues created and estimated.

PHASE 1 — Foundation: Auth, Users, Roles, Admin Shell — Est. 3 weeks

Goal. A secure account system with phone/email sign-in, roles, sessions and an empty-but-secure admin console.

What to build

- Backend: NestJS (or chosen framework) modular monolith, config module, structured logging, health endpoint, global validation and error format.
- Database: migrations for users, user_roles, user_devices, user_sessions, mfa_factors, audit_logs, feature_flags; UUIDv7 keys.
- Auth: Firebase Authentication (phone OTP + email + Google) on the client; backend verifies the Firebase ID token, creates/links the Vexora user and issues its own short-lived access token with refresh rotation.
- RBAC guards plus PostgreSQL Row-Level Security for user-owned data; permission matrix from Section 2 as code and tests.
- Admin shell (Next.js): login, mandatory MFA (TOTP or passkey/WebAuthn), role-based navigation, session list and force-logout.
- Mobile shell (Flutter): onboarding, sign-in, profile, secure token storage, app-version check, theme from the design system.
- Append-only audit log service with hash chaining; rate limiting and abuse protection on OTP and login.
- Account lifecycle: verify, recover, change phone/email, delete/anonymise request.

MCP servers used in this phase

MCP server	How it is used
Dart MCP	Build and analyse the Flutter shell; run widget tests; add packages (firebase_auth, secure storage, dio, riverpod/bloc).
Context7	Current docs for NestJS guards, Firebase Admin token verification, Next.js middleware, Flutter auth packages.
Developer Knowledge	Official Firebase Auth, phone-auth abuse protection and App Check guidance.
Firebase	Configure sign-in providers, authorised domains, App Check (debug tokens for dev), Crashlytics.
Postgres	Verify migrations, indexes, RLS policies and that the audit table rejects UPDATE/DELETE.
Playwright	E2E: admin login with MFA, role-restricted pages, session revoke.
Stitch	Sign-in, onboarding, profile and admin shell screens.
GitHub	One PR per module; CI must pass; code-scanning enabled.
Filesystem	Generate modules, DTOs, migrations and docs.

Security gate

- No custom password storage unless needed; if used, Argon2id with tuned parameters.
- OTP throttling per phone, per IP and per device; CAPTCHA/App Check on send-OTP.
- Refresh-token rotation with reuse detection (reuse revokes the whole session family).
- Admin MFA enforced and tested; verify that a normal user token cannot reach /admin/*.
- Security headers (CSP, HSTS, X-Content-Type-Options, frame protections) on all web apps.

Exit criteria (do not start the next phase until all are true)

[ ] A new user can sign up, sign in on mobile, and see a profile; sessions can be listed and revoked.

[ ] Admin can only sign in with MFA; permission tests green for every role.

[ ] Audit chain verification job passes; CI green with >= 80% coverage on auth/RBAC code.

PHASE 2 Games & Teams Est. 3 weeks

Goal. The player ecosystem: configurable games and modes, player game profiles and teams with rosters.

What to build

- Admin: games, modes, rule templates and versioned scoring definitions (JSON schema validated by the shared package).
- Player: choose favourite games, link in-game ID/UID per game, public/private profile controls.
- Teams: create, invite/accept/leave, roles (captain, manager, player, substitute), roster limits, transfer log and duplicate-identity checks.
- Seasons table and season management (admin).
- Search and filter basics (PostgreSQL full-text and trigram indexes).
- Image upload pipeline for avatars/logos: signed upload URLs, type and size limits, malware scan, image re-encoding.
- Mobile screens: Games, Team list, Team detail, Invite flow, Profile editor.

MCP servers used in this phase

MCP server	How it is used
Stitch	Games hub, team page, invite sheet, profile editor designs.
Dart MCP	Implement and test the screens; add golden tests for key components.
Context7	Docs for the image upload libraries, pagination patterns, ORM relations.
Postgres	Check query plans for team/game lookups; verify unique constraints (one captain, one UID per game).
Playwright	Admin game/mode/scoring-template CRUD flows.
Filesystem	Generate game-definition JSON schemas and seed data.
GitHub	PRs per feature; issue labels by module.

Security gate

- Signed URLs for uploads; reject SVG/script content; strip metadata (EXIF) from images.
- Invite spam limits and block lists; ensure a user cannot add themselves to a team without consent.
- Authorisation tests: only the captain/manager can edit the roster.

Exit criteria (do not start the next phase until all are true)

[ ] Admin can define a game, mode and scoring version; players can create a team and invite members.

[ ] Duplicate in-game UID rule enforced; upload pipeline tested with hostile files.

PHASE 3 Tournament MVP Est. 4 weeks

Goal. Organisers can create tournaments, players can discover and register, and brackets/match schedules are generated.

What to build

- Tournament module with the full state machine from Section 4; invalid transitions rejected and audit-logged.
- Tournament creation wizard (web + mobile organiser flavour) with validation of schedule, capacity, rules version and prize text.
- Discovery: filters (game, date, format, entry type, prize, organiser status), sorting, deep links and QR join links.
- Registration with capacity control (row lock / unique constraint), waitlist, check-in and roster lock.
- Bracket/match engine: knockout, round-robin and league first; byes and seeding; Battle Royale lobby grouping for BR games.
- Organiser dashboard: participants, check-in status, announcements, match list.
- Free tournaments only in this phase (no money flow yet). Admin tournament review/approval queue.

MCP servers used in this phase

MCP server	How it is used
Stitch	Discover, tournament detail, join sheet, organiser create wizard, bracket view.
Dart MCP	Build lists, filters, bracket widgets; run integration tests on low-end device profiles.
Postgres	Concurrency proof: simulate many simultaneous registrations for the last slot; verify indexes for discovery queries.
Playwright	E2E: organiser creates -> admin approves -> player registers -> check-in -> bracket generated.
Context7	Docs for queue/worker library (BullMQ), scheduling, and date/timezone handling.
Filesystem / GitHub	Module scaffolding; PR per sub-feature; milestone tracking.

Security gate

- Server time only for registration windows; client time never trusted.

- Organiser can only edit tournaments they own (tested); changes after publish are versioned and notified.
- Rate limits on registration and tournament creation; abuse test with scripted bots.

Exit criteria (do not start the next phase until all are true)

[ ] A full 16-team free tournament can run from creation to a generated bracket on staging.

[ ] Concurrency test: exactly N winners for N slots, no double registration.

PHASE 4 Results, Evidence, Scoring & Leaderboards Est. 3 weeks

Goal. The competitive loop: submit results with evidence, validate, finalise, score and rank.

What to build

- Match result submission with idempotency keys; keep raw submissions and the adjudicated result separately.
- Evidence upload (screenshots, short video) with SHA-256 hash recorded at upload, signed URLs, size/type limits and antivirus scanning.
- Confirm / counter-submit flow for teams; conflict -> automatic hold and dispute stub (full workflow in Phase 5).
- Scoring engine package (placement + kills + bonuses, tie-breakers) shared by server and client preview; formulas versioned.
- Leaderboard transaction: score rows -> season stats -> cached leaderboard (Redis) with historical snapshots.
- Player dashboard: matches, wins, average placement, points/match, recent form, tournament and team history.
- "Verify this result" page with result hash, evidence hashes and formula version.

MCP servers used in this phase

MCP server	How it is used
Dart MCP	Result submission UI, evidence picker/compressor, dashboard charts; widget and integration tests.
Postgres	Verify scoring writes in a single transaction; check leaderboard queries and snapshot tables.
Playwright	E2E: submit -> confirm -> finalise -> leaderboard updates; conflicting submissions create a hold.
Context7	Docs for file-upload streaming, image/video processing and charting libraries.
Stitch	Result entry, evidence viewer, dashboard and leaderboard screens.
GitHub	Scoring engine in its own package with its own tests and changelog.

Security gate

- Uploaded files re-validated by magic bytes, scanned, stored with random keys, served via short-lived signed URLs.
- Results immutable after finalisation; corrections only via an authorised, audited correction record.
- Mutation tests and property tests on the scoring engine.

Exit criteria (do not start the next phase until all are true)

[ ] Scoring tests match hand-calculated examples for every launch game.

[ ] A finalised result is reproducible from stored inputs and formula version.

PHASE 5 — Trust: Organiser Verification, Reports, Disputes & Moderation — Est. 3 weeks

Goal. Make Vexora safe: only verified organisers host public events, and problems have a fair, auditable path.

What to build

- Organiser application, document upload to a separate encrypted bucket, KYC provider integration where legally required, admin review queue.
- Organiser tiers (Starter / Verified / Partner), public trust indicators and publishing holds triggered by rules.
- Reports (player, team, organiser, result, chat) with categories; report is not a violation until reviewed.
- Dispute workflow: case ID, evidence attach, settlement freeze, moderator decision with reason, appeal window, final decision.
- Sanction ladder (warning, restriction, suspension, ban) with published policy and appeal.
- Ratings and reviews after completed tournaments with sample size and moderation.
- Moderator console: queue, case view, evidence viewer, macros, SLA timers; Security Centre v1.

MCP servers used in this phase

MCP server	How it is used
Stitch	Organiser application, moderator queue and case view, trust badge/profile.
Playwright	E2E for the application -> approval -> publish path; dispute -> decision -> appeal path; permission boundaries for moderators.
Postgres	Confirm KYC documents are in a separate schema/bucket and not readable by ordinary roles; verify case/evidence integrity.
Developer Knowledge	Review Firebase/Google guidance for abuse prevention and App Check enforcement.
Context7	KYC/e-sign provider SDK docs; queue and SLA scheduler docs.
GitHub	Threat-model update PR; labelled security issues.

Security gate

- Moderators see evidence but not KYC documents; KYC access requires a separate role plus logged reason.
- Every moderation decision stores actor, reason, evidence refs and policy clause.
- No automatic bans in high-impact cases; risk signals only prioritise review.

Exit criteria (do not start the next phase until all are true)

[ ] An organiser cannot publish a public tournament until approved.

[ ] A disputed result freezes settlement and ends with an auditable decision; appeal path tested.

PHASE 6 Wallet, Ledger, Redeem Codes & Payments Est. 4 weeks

Goal. A controlled closed-loop credit system. Enable only after the legal/payment review for your launch jurisdictions.

What to build

- Double-entry ledger tables with append-only triggers, idempotency keys and invariant checks (property tests).
- Wallet service: CREDIT, HOLD, RELEASE, REFUND, REWARD, DEBIT/SETTLEMENT, ADJUSTMENT (reason + admin ID + second approver).
- Redeem codes: HMAC-hashed storage, checksum, batches, expiry, rate limits, lock-out and anomaly alerts, encrypted batch export.
- Payment-provider integration with signed webhooks, replay protection and webhook_events de-duplication; client never confirms payment.
- Tournament entry using holds, prize escrow, refunds on cancellation, settlement job.
- Payout request queue (requested -> under review -> approved -> processing -> paid/failed/rejected) with maker-checker approval.
- Daily reconciliation job: ledger vs provider vs payouts, with discrepancy dashboard and alerts.
- Feature flags so credits/payouts can be disabled instantly per region.

MCP servers used in this phase

MCP server	How it is used
Postgres	Verify ledger invariants with SQL checks; inspect locking and isolation behaviour; review slow queries on ledger search.
Playwright	E2E: redeem code twice (second is rejected), registration hold/release, payout approval with two admins.
Context7	Payment SDK and webhook-signature docs; decimal/money handling libraries.
Firebase	Remote Config / feature flags for regional kill-switches; App Check enforced for wallet endpoints.
Developer Knowledge	Play Integrity / App Attest guidance for sensitive calls.
Dart MCP	Wallet screens, biometric gate, transaction history with offline-safe states.
GitHub	Mandatory two-reviewer rule on the wallet/ payouts folders (CODEOWNERS).

Security gate

- Sum of all ledger entries must equal zero; balance can never go negative (DB constraint plus test).
- Money uses NUMERIC/integer minor units, never floating point.
- Redeem-code brute-force tests pass; maker-checker enforced (creator cannot approve own payout).
- Webhook signature, timestamp and duplicate-event tests pass; reconciliation reports zero unexplained differences on test data.

Exit criteria (do not start the next phase until all are true)

[ ] Legal sign-off recorded for each enabled region; otherwise the wallet stays in test/free mode.

[ ] Full cycle works on staging: redeem -> join -> hold -> cancel -> release; and finish -> reward -> payout request -> reconcile.

PHASE 7 Real-time, Push & Live Experience Est. 2 weeks

Goal. Live tournament status, results and notifications delivered instantly and reliably.

What to build

- WebSocket gateway (Socket.IO) with token auth, per-channel authorisation, rate limits and Redis pub/sub for horizontal scale.
- Event catalogue from Section 14 with sequence numbers, reconnect and missed-event replay.
- Firebase Cloud Messaging: topic and device targeting, notification preferences, quiet hours, localisation, delivery tracking and dead-letter queue.
- In-app inbox, deep links from notifications to match/tournament/dispute screens.
- Live tournament board and live leaderboard on mobile and web; organiser announcement channel with moderation.
- Maintenance banner and degraded-mode behaviour (read-only if real-time fails).

MCP servers used in this phase

MCP server	How it is used
Firebase	Configure FCM, topics, notification channels and Crashlytics alerts for real-time failures.
Developer Knowledge	Current Android/iOS notification permission, channel and background-delivery rules.
Dart MCP	WebSocket client, reconnect logic, notification handling; tests with simulated flaky networks.
Playwright	Multi-browser E2E: two users see the same live update; unauthorised channel join is denied.
Context7	Socket.IO adapter, Redis pub/sub and FCM SDK docs.
Postgres	Outbox pattern table so events are published only after the database commit.

Security gate

- Per-connection and per-user message rate limits; payload size limits.
- A user cannot subscribe to another user's wallet or another tournament's private channel.
- No sensitive data (balances, KYC, room codes) in push payloads; send IDs and fetch securely.

Exit criteria (do not start the next phase until all are true)

[ ] Result finalisation updates the leaderboard on two devices within seconds.

[ ] Notification delivered on Android and iOS test devices; reconnect replay verified.

PHASE 8 Analytics, Achievements & Support Desk Est. 3 weeks

Goal. Turn data into player identity, organiser insight and operational control.

What to build

- Player analytics: trends per game/season, role history, consistency; organiser analytics: registrations, check-in, completion, disputes, ratings; admin analytics: DAU/MAU, retention, liability, payout time, fraud cases.
- Reporting tables/materialised views refreshed by workers; no heavy analytics on the primary tables during live events.
- Achievements and badges engine (Section 42) with verified-source rules.
- Support ticket system with SLAs, categories, canned replies, escalation to finance/moderation, CSAT survey.
- Firebase Analytics events plan and funnel dashboards; privacy-respecting event naming without personal data.
- Admin dashboards finished with charts and export (CSV/PDF) under role control.

MCP servers used in this phase

MCP server	How it is used
Stitch	Dashboard, charts and ticket desk designs.
Postgres	Design and test materialised views/indexes; EXPLAIN every dashboard query; read-replica readiness.
Dart MCP	Dashboard charts, share cards, achievements UI.
Playwright	Admin dashboard and ticket flows; export permission tests.
Firebase	Analytics events, conversion funnels, Remote Config experiments.
Context7	Charting library and cron/worker docs.

Security gate

- Analytics exports masked for personal data; role-limited and logged.
- Retention rules applied to event data; consent honoured for analytics.

Exit criteria (do not start the next phase until all are true)

[ ] Every dashboard metric has a documented definition and a test against known data.

[ ] Support SLAs visible and breach alerts working.

PHASE 9 Hardening: Security, Load, DR & Store Preparation Est. 3 weeks

Goal. Prove the system is secure, fast and recoverable before real users arrive.

What to build

- Independent penetration test (or at least a structured internal one) against API, apps, admin and storage; fix critical/high issues.
- OWASP ASVS and OWASP Mobile (MASVS) checklists completed; threat model reviewed.
- Load tests: tournament-start spike, leaderboard refresh, WebSocket fan-out, redeem-code endpoint under attack.
- Backup, point-in-time recovery and a full restore drill (database + object-storage references); define RPO/RTO.
- Observability: dashboards, alerts, error budgets, log redaction review (Section 46).
- App-store preparation: privacy labels/data-safety form, permissions review, screenshots, age rating, account-deletion flow, review notes.
- Accessibility audit, localisation review, low-end device performance pass.

MCP servers used in this phase

MCP server	How it is used
Playwright	Full regression suite plus visual and accessibility checks on every release candidate.
Dart MCP	Analyse, profile and test release builds; dependency and lint clean-up; size/performance checks.
GitHub	Code-scanning, Dependabot, secret-scanning alerts triaged; release branch and tagged RC.
Postgres	Index review, vacuum/bloat checks, restore verification queries on the recovered database.
Firebase	Enforce App Check, review security rules, Crashlytics stability thresholds.
Developer Knowledge	Final check against current Google Play and Firebase policy/requirements.

Security gate

- Zero open critical/high vulnerabilities; medium issues have owners and dates.
- Secrets rotated; production access reviewed; least-privilege audit signed.
- Restore drill completed with timing recorded.

Exit criteria (do not start the next phase until all are true)

[ ] Load test targets met (define them: for example p95 API < 300 ms at expected peak).

[ ] Release candidate passes the complete E2E suite on staging; store submission checklist complete.

PHASE 10 **Pilot & Controlled Launch** **Est. 3 weeks+**

Goal. Release to a small group, learn, fix, then widen gradually.

What to build

- Closed beta via Firebase App Distribution / TestFlight / Play internal testing with 20-50 players and 2-3 verified organisers.
- Run 2-3 complete pilot tournaments end-to-end with a moderator on standby and a live incident channel.
- Staged rollout: percentage-based release, feature flags, regional enablement, monitoring of crash-free rate, disputes, payment failures.
- Publish Terms, Privacy, Refund, Dispute and Code of Conduct; enable the support desk and status page.
- Collect feedback, review dispute and completion metrics weekly; decide on enabling paid features only after the legal review.
- Post-launch retro and the next-quarter roadmap (Section 41 and 42 items).

MCP servers used in this phase

MCP server	How it is used
Firebase	App Distribution, staged rollout flags in Remote Config, Crashlytics monitoring.
Developer Knowledge	Production-readiness and store-policy checks.
GitHub	Release notes, hotfix branches, issue triage board for pilot feedback.
Playwright	Production smoke tests (read-only, test accounts) after each deployment.
Postgres	Read-only checks on staging copies of anonymised data; no direct production access from MCP.

Security gate

- Incident-response runbook rehearsed; on-call and escalation contacts confirmed.
- Kill-switches for wallet, payouts, registration and publishing tested.
- Data-subject request process (access/delete) tested end-to-end.

Exit criteria (do not start the next phase until all are true)

[ ] Pilot tournaments complete without unresolved financial discrepancies.

[ ] Crash-free sessions above target; dispute rate and resolution time within thresholds.

40. MCP-to-Phase Matrix

Use this as the quick reference for which MCP server to prioritise in each phase (main = primary driver, used = supporting, blank = not needed).

Server	0	1	2	3	4	5	6	7	8	9	10
Dart MCP	main	main	main	main	main		used	main	used	main	
Context7	used	main	used	used	used	used	main	used	used		
Filesystem	main	main	main	main	used	used	used	used	used	used	
Stitch	main	main	main	main	main	used			main		
Developer Knowledge	used	main				used	used	main		main	main
Firebase	main	main					used	main	used	main	main
Playwright	used	main	used	main	main	main	main	main	used	main	used
GitHub	main	main	main	main	main	main	main	main	main	main	main
Postgres	used	main	main	main	main	main	main	used	main	main	used

41. Extended Feature Set (Vexora Additions)

These features are additions to the original blueprint. They are grouped by priority so they can be slotted into later releases without disturbing the MVP.

Player features

- Player profile card and shareable stat card (image) with a verification QR linking to the verified-stats page.
- Scrims and practice rooms: free, unranked lobbies with the same room/lobby workflow.
- Looking-for-team (LFT) board and team recruitment posts with role/rank filters and anti-spam limits.
- Match reminders (calendar export / add to calendar), check-in countdown and lobby-ready alerts.
- Result replay checklist: guided evidence capture so players know exactly which screenshots to submit.
- Follow organisers, teams and games; personalised "for you" feed using simple rules first.
- Friends list, party invites and a safe in-app chat (reports, mute, block, profanity filter; no phone numbers shown).
- Fair-play pledge at sign-up and a visible Fair-Play score computed only from confirmed outcomes.
- Spectator mode (live bracket, live leaderboard) and a public tournament page that can be shared without signing in.
- Account safety centre: active sessions, trusted devices, recovery options, login history.

Organiser features

- Tournament templates and clone-previous-tournament.
- Co-organisers and match referees with scoped permissions.
- Bulk result entry (table/CSV) with validation and a scoring simulator.
- Sponsor/partner banners with moderation.
- Automated check-in, no-show handling and waitlist promotion; auto-reminders.
- Organiser wallet view (fees, escrow status, settlements) with downloadable statements.
- Post-tournament report: registrations, completion, disputes, ratings, time to finalise.
- Roadmap: streaming links, stage/broadcast overlays and caster tools.

Admin / operations features

- Feature flags and regional kill-switches (wallet, payouts, registration, publishing).
- Maker-checker approvals for payouts, adjustments, code batches and role changes.
- Case management timeline linking reports, disputes, tickets, ledger entries and audit events for one user.
- Fraud graph view (accounts linked by device/payment/IP hash) for reviewers, with privacy controls.
- Policy engine: sanctions and holds configured as policy, not hard-coded.
- Scheduled reports to finance/compliance; export with watermarking and logging.
- Admin impersonation is NOT allowed; use a read-only "view as" simulator with masked data instead.

Platform features

- Multi-language and regional formatting (English, Hindi, Odia and others); right-to-left readiness.
- Seasons, ladders, qualifiers and open/closed qualifier-to-finals pipelines.
- Public developer API (read-only) and webhooks for approved partners, after launch.
- Referral programme with anti-abuse rules (reward only after verified activity).
- Referee/anti-cheat integration points for supported games and publisher-approved result APIs.
- Creator tools: highlight clips, tournament recap pages, social share templates.

42. Gamification & Retention

Mechanic	Design rule
Achievements and badges	Awarded only from verified, finalised results; each badge shows the source tournament and date.
Seasons	Fixed start/end, snapshot of rankings, archived history; rewards are cosmetic first.
Streaks and missions	Participation and fair-play focused (check in on time, complete matches), not spend-based.
Titles and profile frames	Earned cosmetics; no pay-to-win advantage.
Leaderboards	Per game and season, plus friends/region/team boards; rank formulas published.
Reputation tiers	Based on completion, fair play and review quality; shown with sample size.

Responsible design. Avoid dark patterns: no manipulative countdowns for spending, no hidden odds, easy self-exclusion/limit tools, and clear age restrictions. Reward fair play and participation, not spending.

43. Organiser Productivity Tools

- Match-day checklist: lobby created, room details released on schedule, check-in verified, evidence collected, result submitted.
- Room code handling: release at a set time to checked-in players only; mask in screenshots; expire after use.
- Incident log for every tournament (late start, disconnects, rule clarifications) visible to moderators.
- Auto-generated public rules page and tournament announcement from the structured data (no manual retyping).
- Co-pilot suggestions using AI (draft rules, announcements, FAQs) clearly labelled and always reviewed by the organiser; AI never edits results or money.

44. Vexora Brand & Design System (Stitch-driven)

- Principles: fast, competitive, trustworthy; clear information hierarchy; accessible contrast (WCAG AA); dark mode first with a light theme.
- Design tokens: colours, typography scale, spacing, radius, elevation, motion; exported to Flutter ThemeData and CSS variables so web and mobile match.
- Component library: cards (tournament, team, match), bracket, leaderboard row, status chips (state machine colours), evidence tile, wallet transaction row, empty states, error states.
- Trust UI patterns: verified badge, evidence hash tick, "published rules" link and formula version on every result.
- Process: describe screen in Stitch -> review -> export design/code -> implement in Flutter/Next.js using the tokens -> compare against the design with Playwright screenshots.
- Keep a Vexora UI kit page in the web app (Storybook-style) that shows every component in both themes.

45. DevOps, CI/CD & Environments

Environment	Purpose	Data / access
Local	Developer machine via Docker Compose	Seed data only; MCP servers allowed
Dev / Preview	Per-PR preview deployments	Synthetic data; MCP read-only Postgres allowed
Staging	Mirror of production for release testing	Anonymised/synthetic data; test payment keys
Production	Real users	No direct MCP/AI-tool access; human-run, audited access only

CI/CD pipeline (GitHub Actions)

```
on pull request:
  lint + format -> unit tests -> build -> DB migration dry-run -> integration tests
  -> SAST (CodeQL/Semgrep) -> dependency scan -> secret scan -> container scan
  -> Playwright E2E on preview -> Flutter analyze/test -> coverage gate
on merge to main:
  build images -> sign images + SBOM -> deploy to staging -> smoke tests
on release tag:
  manual approval (2 people for money modules) -> canary/rolling deploy
  -> health checks -> auto-rollback
```

- Infrastructure as code (Terraform) and reproducible Docker images; no manual console changes in production.
- Database migrations are forward-only, reviewed, run with a dry-run, and backed by a rollback/compensation plan.
- Blue/green or canary deployment so a bad release can be rolled back in minutes; mobile kill-switch via Remote Config.
- Dependabot/Renovate for dependencies, with CI gating; pinned versions and a lock file for every package manager.
- Mobile: Fastlane (or Codemagic) builds, code signing in a secure store, staged rollout on Play and phased release on App Store.

46. Observability, SLOs & Incident Response

Area	What to track
Logs	Structured JSON, correlation ID per request, personal data redacted, retained per policy.
Metrics	API latency/error rate, WebSocket connections, queue depth, DB connections/locks, payment webhook delay, notification delivery rate.
Tracing	OpenTelemetry across API, workers and DB for slow-request analysis.
Errors	Sentry (backend/web) and Firebase Crashlytics (mobile) with release tracking.
Business alerts	Ledger imbalance, reconciliation difference, abnormal redeem-code failures, sudden dispute spike, payout backlog.
Synthetic checks	Playwright smoke tests on login, tournament list, join and wallet read every few minutes.

SLO (starting targets)	Value
API availability	99.9% monthly
API p95 latency (normal load)	< 300 ms
Result-to-leaderboard update	< 5 seconds
Push delivery (accepted by FCM)	> 99%
Payout request first review	Within 24 hours
Support first response	Within 4 hours (safety: 1 hour)

Incident response

- Severity levels (SEV1 money/safety/data, SEV2 live tournament impact, SEV3 minor) with owners and response times.
- Runbooks: database failover, stuck payouts, webhook outage, leaked secret (rotate immediately), compromised admin account, evidence storage outage, DDoS.
- Status page and in-app banner; user communication templates; post-incident review within 5 working days with action items tracked in GitHub.
- Data-breach procedure aligned with legal duties to notify the regulator and affected users within the legally required timelines.

47. Extended Security Architecture

This section expands Section 18 and Section 19 into a layered, zero-trust model. The goal is that no single mistake (a leaked token, a rogue admin, a vulnerable dependency) can silently damage users, money or results.

Layered defence

Layer	Controls
Edge	CDN/WAF with managed rules, DDoS protection, bot management, geo/IP reputation signals (risk only), TLS 1.2+ with HSTS preload, rate limiting by IP, user and device.
Identity	Firebase Auth + backend token verification; short-lived access tokens; refresh rotation with reuse detection; passkeys/TOTP for admins; step-up auth for wallet/payout; new-device alerts.
Application	Schema validation on every input; deny-by-default authorisation; object-level checks (no IDOR); CSRF protection; strict CORS; security headers; safe file handling; SSRF protection on any URL fetch.
Data	Encryption in transit and at rest; field-level encryption for KYC/PII; separate storage and keys for KYC documents; tokenisation of payment data (never store card data); pseudonymised analytics.
Database	Least-privilege DB roles per service; RLS; append-only ledger/audit; no shared superuser; parameterised queries; encrypted backups; network-isolated.
Infrastructure	Private networking, security groups, no public DB; secrets manager with rotation; hardened containers (non-root, read-only filesystem); signed images; IaC reviews.
Mobile	Secure keystore/Keychain, certificate pinning (with a safe rotation plan), Play Integrity/App Attest, obfuscation, no secrets in the app, jailbreak/root signals into risk score, secure deep-link validation.
Operations	Break-glass access with expiry, just-in-time privileges, mandatory MFA, admin session recording, quarterly access reviews.

Tamper-evident audit log

- Each audit record stores a hash of the previous record (hash chain); a scheduled job signs a Merkle root checkpoint and stores it outside the primary database.
- Verification job runs daily and on demand; any break raises a SEV1 alert.
- Audit entries record actor, role, IP hash, device, action, entity, before/after, reason and request ID.
- Auditors have read-only access; nobody (including Super Admin) can delete or edit audit rows.

Evidence integrity and result trust

- SHA-256 of every evidence file at upload; hash stored in the database and in the audit chain.
- Original files are never overwritten; thumbnails/derivatives are separate objects.
- Server timestamps only; EXIF is read for review but never trusted.
- Optional watermarking of room codes and lobby screens per player to trace leaks.
- Result submission signed by the submitting user session; changes create new versions, never edits.

Wallet and payout security

- Maker-checker for adjustments, code batches and payouts; high-value thresholds require two approvers plus step-up auth.

- Velocity limits and cooling-off periods on new accounts, new devices and new payout destinations.
- Payout destination changes trigger notifications and a delay window.
- Dual control over reconciliation overrides; every discrepancy tracked to closure.
- Kill-switches per region and per feature, controllable from the admin console and Remote Config.

Secure SDLC

- Threat modelling (STRIDE) for each new module; security acceptance criteria in every ticket.
- Mandatory code review; CODEOWNERS for wallet, auth and admin code; signed commits.
- SAST, DAST, dependency/SCA, secret and container scanning in CI; SBOM per release.
- Annual (and pre-launch) penetration testing; bug-bounty / responsible-disclosure page (security.txt) after launch.
- Security training for the team; "no real data in dev" rule; production data never copied unmasked.
- OWASP ASVS (web/API) and OWASP MASVS (mobile) used as the verification standard.

OWASP Top-10 mapping (quick check)

Risk	Vexora control
Broken access control	RBAC + object-level checks + RLS + automated permission-matrix tests
Cryptographic failures	TLS, strong hashing, KMS-managed keys, field-level encryption, no custom crypto
Injection	ORM/parameterised queries, schema validation, output encoding, CSP
Insecure design	Threat models, state machines, idempotency, maker-checker
Security misconfiguration	IaC, hardened images, config scanning, least privilege
Vulnerable components	SCA, Dependabot, pinned versions, SBOM
Auth failures	MFA, rotation, throttling, session management, App Check
Integrity failures	Signed builds/images, hash-chained audit, webhook signature checks
Logging failures	Central logs, alerts, redaction, tamper-evidence
SSRF / uploads	Allow-lists, no server-side fetch of user URLs, malware scan, re-encoding

48. Privacy & Data Protection

- Data minimisation: collect only what each feature needs; mark each field mandatory/optional (Section 31).
- Data map and register of processing: what is collected, why, where stored, who can access, how long kept.
- Consent management: clear, granular, revocable consent for analytics, notifications, location and marketing; separate guardian consent for minors where required.
- User rights: export my data, correct my data, delete my account; workflows for each with identity checks and SLAs.
- Retention schedule: for example evidence 12 months after case closure, chat 90 days, logs 12 months, KYC and financial records as required by law, then delete/anonymise.
- Anonymisation/pseudonymisation for analytics; hashing of IP/device identifiers with rotating salts.
- Vendor management: list of processors (Firebase, payment, SMS, KYC, hosting) with data-processing agreements and region choices.
- Cross-border transfer review; data-residency decisions recorded as ADRs.
- Privacy by default for profiles: minimal public data; no public phone/email/location.
- Align with India's DPDP Act and any other applicable law (confirm with counsel).

49. Performance & Scalability

- Start as a modular monolith with background workers; scale horizontally behind a load balancer; keep the API stateless.
- PostgreSQL: right indexes, connection pooling (PgBouncer), read replicas for analytics, partitioning for large tables (ledger, events, notifications) by time.
- Redis for caching leaderboards, rate limits and pub/sub; cache invalidation tied to result finalisation.
- Queues for emails, push, image processing, scoring and reconciliation; retry with backoff and dead-letter queues.
- CDN for static assets and signed-URL media; image resizing and lazy loading on mobile.
- Mobile performance budget: cold start < 2.5 s on a low-end Android device, 60 fps scrolling lists, small app size, offline-friendly caches.
- Tournament-start spike plan: pre-warm caches, queue admissions, shed non-critical load, protect wallet and results endpoints first.
- Capacity tests before every major tournament season; autoscaling rules and cost alerts.

50. Risk Register

Risk	Impact	Mitigation
Legal/regulatory change on paid entry or prizes	Launch blocked or forced shutdown of money features	Free-first launch, feature flags, regional kill-switch, legal review, ADR log
Organiser takes entry fees and disappears	User harm, brand damage	Verification, escrow, holds, refunds, tier limits, public history
Result manipulation / collusion	Loss of trust	Evidence hashing, dual confirmation, anomaly detection, disputes
Redeem-code leak or insider abuse	Financial loss	HMAC storage, one-time use, encrypted export, maker-checker, alerts
Account takeover	Wallet and identity harm	MFA/step-up, device trust, alerts, session revocation
Scope creep	Delays	MVP boundary (Section 28), phase gates, backlog grooming
AI-tool misuse (MCP over-permissioned)	Data or repo damage	Least privilege, read-only DB, branch protection, human approval (Section 38)
Single-developer dependency	Bus-factor risk	Docs, ADRs, runbooks, CI, pair reviews
Publisher/game policy conflict	Games can be restricted	Check each game's terms, avoid trademark misuse, get permissions where needed
Platform outage during a big event	Reputation loss	Load tests, SLOs, runbooks, degraded mode, status page

51. Sprint Cadence & Working Agreement

- Two-week sprints inside each phase; a phase usually contains 1-3 sprints.
- Definition of Ready: user story, acceptance criteria, API/DB notes, design (Stitch link), security notes.
- Definition of Done: code reviewed, tests added and green, docs updated, migration reviewed, accessibility checked, security checklist ticked, demo on staging.
- Branching: trunk-based with short-lived branches named phase-N/feature-name; Conventional Commits; squash merge.
- Weekly demo, fortnightly retrospective, monthly security review and dependency update day.
- ADR for any architectural decision; runbook for any operational procedure.
- Keep CLAUDE.md current: current phase, module map, coding standards, security rules and the list of enabled MCP servers.

52. Final Definition of Done for Vexora v1

[ ] All eleven phases passed their exit criteria and security gates

[ ] Legal, privacy and payment reviews completed and recorded for every enabled region

[ ] Ledger invariants proven by tests and daily reconciliation reports clean

[ ] Audit-chain verification, backup restore and incident-response drills completed

[ ] Penetration test passed with no open critical/high findings

[ ] Pilot tournaments completed with acceptable dispute rate and user feedback

[ ] Store listings approved; account deletion and privacy disclosures in place

[ ] Dashboards, alerts, status page and support desk live

[ ] Documentation complete: API spec, ADRs, runbooks, policies, user help centre

[ ] Roadmap for the next release agreed (Sections 41 and 42)

Final note. Vexora is strongest when it launches small and trustworthy: free tournaments first, verified organisers, transparent rules and results, a tamper-evident audit trail, and money features enabled only after legal review. Build phase by phase, let each MCP server do the job it is best at, and never skip a security gate.